

GETTER AND SETTER IN DART

Introduction

In this section, you will learn about **Getter and Setter** in dart with the help of examples.

Getter And Setter

Getter and **Setter** provide explicit read and write access to an object properties. In dart, **get** and **set** are the keywords used to create getter and setter. Getter read the value of property and act as **accessor**. Setter update the value of property and act as **mutator**.

Info

Note: You can use same name for **getter** and **setter**. But, you can't use same name for **getter**, **setter** and **property name**.

Use Of Getter and Setter

- Validate the data before reading or writing.
- Restrict the read and write access to the properties.
- Making the properties read-only or write-only.
- Perform some action before reading or writing the properties.

Example 1: Getter And Setter In Dart

In this example below, there is a class named **Student** with three private properties **_firstName**, **_lastName** and **_age**. There are two getters **fullName** and **age** to get the value of the properties. There are also three setters **firstName**, **lastName** and **age** to update the value of the properties. If **age** is less than 0, it will throw an error.

```
class Student {
  // Private Properties
  String? _firstName;
  String? _lastName;
  int? _age;

  // Getter to get full name
  String get fullName => this._firstName! + " " + this._lastName!;

  // Getter to read private property _age
  int get age => this._age!;

  // Setter to update private property _firstName
  set firstName(String firstName) => this._firstName = firstName;

  // Setter to update private property _lastName
  set lastName(String lastName) => this._lastName = lastName;

  // Setter to update private property _age
  set age(int age) {
    if (age < 0) {
      throw new Exception("Age can't be less than 0");
    }
    this._age = age;
  }
}

void main() {
  // Create an object of Student class
  Student st = new Student();
  // setting values to the object using setter
  st.firstName = "John";
  st.lastName = "Doe";
  st.age = 20;
  // Display the values of the object
  print("Full Name: ${st.fullName}");
  print("Age: ${st.age}");
}
```

[Show Output](#)

[Run Online](#)

Example 2: Getter And Setter In Dart

In this example below, there is a class named **BankAccount** with one private property **_balance**. There is one getter **balance** to read the value of the property. There are methods **deposit** and **withdraw** to update the value of the **_balance**.

```
class BankAccount {
    // Private Property
    double _balance = 0.0;

    // Getter to read private property _balance
    double get balance => this._balance;

    // Method to deposit money
    void deposit(double amount) {
        this._balance += amount;
    }

    // Method to withdraw money
    void withdraw(double amount) {
        if (this._balance >= amount) {
            this._balance -= amount;
        } else {
            throw new Exception("Insufficient Balance");
        }
    }
}

void main() {
    // Create an object of BankAccount class
    BankAccount account = new BankAccount();
    // Deposit money
    account.deposit(1000);
    // Display the balance
    print("Balance after deposit: ${account.balance}");
    // Withdraw money
    account.withdraw(500);
    // Display the balance
    print("Balance after withdraw: ${account.balance}");
}
```

[Show Output](#)

[Run Online](#)

When To Use Getter And Setter

- Use getter and setter when you want to restrict the access to the properties.
- Use getter and setter when you want to perform some action before reading or writing the properties.
- Use getter and setter when you want to validate the data before reading or writing the properties.
- Don't use getter and setter when you want to make the properties read-only or write-only.

Video

Watch our video on getter and setter in Dart.